

Oshin Style Studio — Technical Documentation

Virtual Try-On & Styling System

June 29, 2026

CONTENTS

Oshin Style Studio — Virtual Try-On & Styling System	
1. What this is	
2. Architecture at a glance	
3. Tech stack	
4. External APIs & services	
4.1 Fashn.ai — virtual try-on (core engine)	
4.2 Supabase — database (PostgreSQL)	
4.3 Cloudinary — image hosting	
4.4 AILab Tools (via RapidAPI) — hairstyle try-on	
4.5 Shopify	
5. Backend modules (backend/)	
6. API endpoints	
7. The recommendation engine	
8. Catalog / data pipeline	
9. Frontend & embeddable widgets (widget/)	
The try-on modal flow (per product)	
Shopify install (2 script tags in theme.liquid, before </body>)	
10. Lead capture & customer data	
11. Deployment	

- 12. Environment variables (.env)
- 13. Running locally
- 14. Cost structure
- 15. Operator cheat-sheet

Oshin Style Studio — Virtual Try-On & Styling System

Complete, up-to-date documentation of the product: what it is, what it uses, how it works, and how to operate it.

Last updated: 2026-06-29 · Status: deployed & live on Vercel

1. What this is

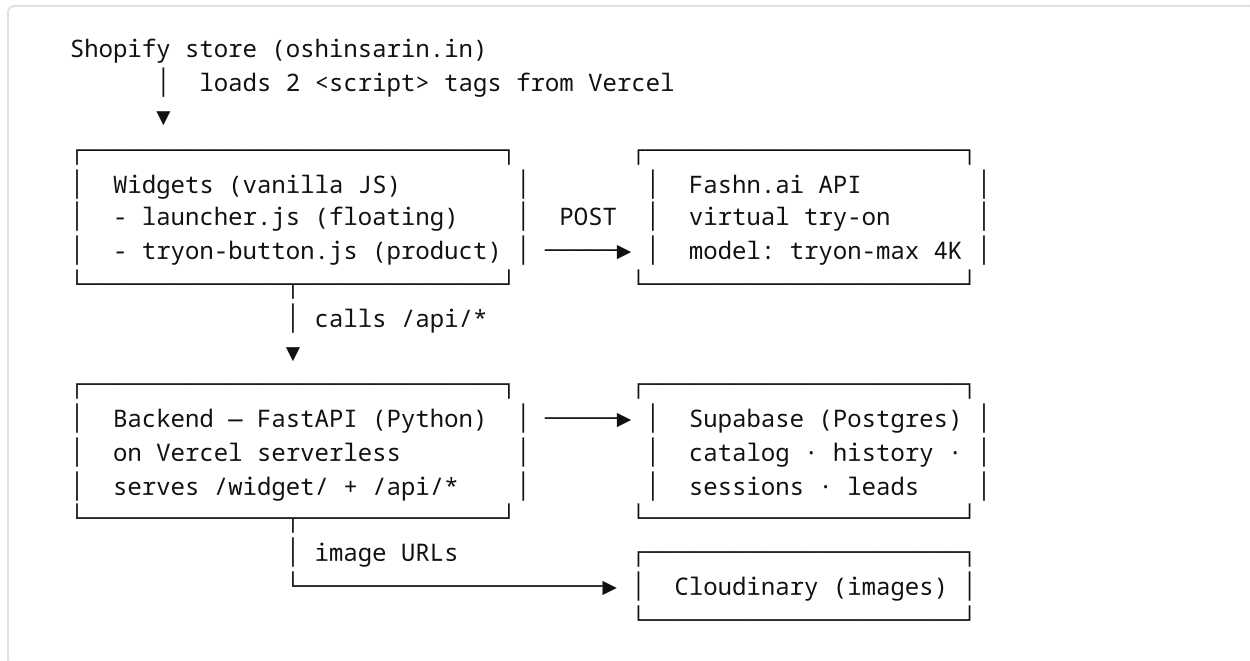
A virtual try-on and styling experience for the fashion brand **Oshin Sarin** (oshinsarin.in, a Shopify store). Shoppers can:

1. Take a **style quiz** and get outfit **recommendations** matched to their body type, occasion, and taste.
2. Upload a photo and **virtually try on** any outfit — AI dresses them in the garment, in 4K.
3. **Save their email** to receive the look — turning try-ons into real leads.
4. Browse the catalog and keep a **history** of their try-ons.

It ships in two forms:

- A **full Style Studio web app** (quiz → recommendations → try-on).
 - **Two embeddable widgets** for the Shopify store: a sitewide floating launcher and a per-product “Virtual Try-On” button.
-

2. Architecture at a glance



3. Tech stack

Layer	Technology
Backend	Python 3, FastAPI + Uvicorn
Database	Supabase (managed PostgreSQL)
Image hosting	Cloudinary
Virtual try-on AI	Fashn.ai (tryon-max model, 4K)
Hairstyle try-on	AILab Tools API (via RapidAPI)
Frontend / widgets	Vanilla JavaScript, HTML, CSS (no framework)
Hosting / deploy	Vercel (serverless Python)
Source control	GitHub — KB-Ajmera/oshin-style-studio
Recommendation engine	Designer-intelligence matching engine
Lead export	Excel (.xlsx) via openpyxl

Live URL: [https://oshin-style-studio-kb-ajmeras-projects.vercel.appSMARTPANTS PRESERVED 258SMARTPANTS PRESERVED 259](https://oshin-style-studio-kb-ajmeras-projects.vercel.appSMARTPANTS%20PRESERVED%20SMARTPANTS%20PRESERVED%20)

4. External APIs & services

4.1 FASHN.AI — VIRTUAL TRY-ON (CORE ENGINE)

- **What:** takes a person's photo + a garment image, returns a photoreal image of the person wearing the garment.
- **Model:** `tryon-max`, `resolution: 4k`, `generation_mode: quality`, `PNG`.
- **Endpoints used:**
 - `POST https://api.fashn.ai/v1/run` — start a try-on (returns a prediction id)
 - `GET https://api.fashn.ai/v1/status/{id}` — poll for the result
 - `GET https://api.fashn.ai/v1/credits` — check credit balance
- **Auth:** `Authorization: Bearer <FASHN_API_KEY>`
- **Code:** `backend/tryon.py`

4.2 SUPABASE — DATABASE (POSTGRESQL)

- **Stores:** product **catalog**, try-on **history**, visitor **sessions**, and captured email **leads**.
- **Connection:** Postgres via pooler (`backend/db.py`) using `psycopg`.
- **Tables:** `products`, `sessions`, `tryon_history`, `comparisons`, `leads`.

4.3 CLOUDINARY — IMAGE HOSTING

- Hosts all product/garment images served to the widget and sent to Fashn.ai.
- **Code:** `backend/upload_to_cloudinary.py`

4.4 AILAB TOOLS (VIA RAPIDAPI) — HAIRSTYLE TRY-ON

- Powers the optional hairstyle feature; makeup looks are presets.
- **Code:** `backend/beauty.py`

4.5 SHOPIFY

- The store. Product **handles** map 1:1 to catalog **SKUs**, so the per-product button knows which garment to try on. Integration is 2 `<script>` tags in the theme — no Shopify app required.
-

5. Backend modules (`backend/`)

<u>File</u>	<u>Responsibility</u>
<code>main.py</code>	FastAPI app, all routes, CORS, security middleware
<code>config.py</code>	Env vars + Fashn.ai URLs
<code>db.py</code>	Supabase Postgres connection helper
<code>catalog.py</code>	Loads the product catalog from Supabase into memory
<code>questionnaire.py</code>	The style quiz + designer-intelligence recommendation matching
<code>tryon.py</code>	Fashn.ai try-on: image preprocessing, start, poll
<code>beauty.py</code>	Hairstyles (AIIab) + makeup presets
<code>history.py</code>	Try-on history, comparisons, and email leads
<code>security.py</code>	Rate limiting, input sanitization, security headers, CSP
<code>merge_metadata.py</code>	Merges Oshin's designer metadata Excel → catalog JSON
<code>migrate_products.py</code>	Pushes the catalog JSON into Supabase
<code>upload_to_cloudinary.py</code>	Uploads product images to Cloudinary
<code>view_leads.py</code>	Prints captured leads (email + profile + outfit)
<code>export_leads.py</code>	Exports all leads to <code>data/leads.xlsx</code>

6. API endpoints

<u>Method</u>	<u>Path</u>	<u>Purpose</u>
<u>GET</u>	<u>/api/health</u>	<u>Health check</u>
<u>GET</u>	<u>/api/questionnaire</u>	<u>The style quiz definition</u>
<u>POST</u>	<u>/api/preferences</u>	<u>Save a visitor's quiz answers</u>
<u>GET</u>	<u>/api/recommendations</u>	<u>Ranked outfit recommendations</u>
<u>GET</u>	<u>/api/categories</u>	<u>Catalog categories</u>
<u>GET</u>	<u>/api/outfits</u> · <u>/api/outfits/{id}</u>	<u>Catalog list / single product</u>
<u>POST</u>	<u>/api/tryon</u> · <u>GET</u> <u>/api/tryon/status/{id}</u>	<u>Start / poll a try-on</u>
<u>POST</u>	<u>/api/lead</u>	<u>Save a captured email lead</u>
<u>GET</u>	<u>/api/hairstyles</u> · <u>/api/makeup</u>	<u>Beauty options</u>
<u>GET/POST/DELETE</u>	<u>/api/history*</u>	<u>Try-on history</u>
<u>GET/POST</u>	<u>/api/comparisons*</u>	<u>Look comparisons</u>

7. The recommendation engine

- File: [backend/questionnaire.py](#)
- A **designer-intelligence matching engine**. Each outfit is matched against the shopper's quiz answers across weighted style signals: occasion (5), style vibe (4), body shape (4), color palette (3), silhouette (3), fabric (2), with budget filtering. Outfits are ranked best-match first.
- The matching is powered by **Oshin's own designer metadata** (occasion, style vibe, color family, silhouette, body types, fit notes) encoded into the catalog — so recommendations feel like a personal consultation with the designer.

8. Catalog / data pipeline

1. [Oshin fills designer metadata in `Oshin-Shopify-Metadata-FILL-IN.xlsx`](#).
2. [python backend/merge_metadata.py "<sheet>.xlsx"](#) — normalizes + merges into [data/shopify_catalog.json](#).
3. [python backend/migrate_products.py](#) — loads all products into Supabase.
4. [The app loads the catalog from Supabase at startup.](#)

[Current catalog: ~213 products \(24 linesheet + 191 Shopify\) with full designer metadata.](#)

9. Frontend & embeddable widgets ([widget/](#))

File	What it is
index.html , app.js , styles.css	The full Style Studio web app
launcher.js	Floating “Virtual Try-On” pill (sitewide). Opens the full Style Studio in a full-screen overlay on the host site.
tryon-button.js	Per-product “Virtual Try-On This Outfit” button. Detects the Shopify product and runs a try-on for that exact outfit.
body-outline.png	Full-body reference figure shown in the upload guidance.

THE TRY-ON MODAL FLOW (PER PRODUCT)

1. **Upload** — two columns: photo dropzone on the left; on the right, a body outline figure + 4 photo rules (“Full body visible”, “Stand straight”, “Good lighting”, “Avoid cropped/blurry”). [The same guidance appears on the app’s “Your Photo” page.](#)
2. **Preview** — [the uploaded photo, a disclaimer \(“For best output, upload a full body image”\), and “Try it on”.](#)
3. **Result** — [the generated image, the disclaimer, **Add to Cart**, download, and an **email capture box**.](#)

SHOPIFY INSTALL (2 SCRIPT TAGS IN `THEME.LIQUID`, BEFORE `</BODY>`)

```
<script src="https://oshin-style-studio-kb-ajmeras-projects.vercel.app/widget/launcher.js" defer></script>
{%- if template contains 'product' -%}
  <script src="https://oshin-style-studio-kb-ajmeras-projects.vercel.app/widget/tryon-button.js"
    data-api="https://oshin-style-studio-kb-ajmeras-projects.vercel.app"
    defer></script>
{%- endif -%}
```

Installed safely on a **duplicated theme**, previewed, then published. The scripts are served from Vercel, so the live store always loads the latest version.

10. Lead capture & customer data

Turns anonymous try-ons into named leads.

- After a try-on, the result screen shows an optional **email field** (“Love it? Get this look in your inbox”).
- On submit, `POST /api/lead` validates the email and stores it in the Supabase **leads** table, linked to the visitor’s session.
- Because it’s linked to the session, each lead carries the visitor’s **style profile** (body shape, size, occasion, style, color, fabric) and the **outfit they tried**.

View / export leads (run locally):

```
python backend/view_leads.py      # prints leads to the terminal
python backend/export_leads.py    # writes data/leads.xlsx
```

`data/leads.xlsx` columns: Email · Date Captured · Outfit Tried · Body Shape · Size · Height · Occasion · Style Vibe · Colors · Fabric · Budget.

Lead emails are kept private — viewable only via these local tools, never exposed through a public URL.

11. Deployment

- **Platform:** Vercel (serverless Python via `api/index.py`, see `vercel.json`).
- **Repo:** `github.com/KB-Ajmera/oshin-style-studio`, branch `main`.
- **Auto-deploy:** pushing to `main` triggers a Vercel build automatically.
- **Production URL:** `https://oshin-style-studio-kb-ajmeras-projects.vercel.appSMARTPANTS PRESERVED 709SMARTPANTS PRESERVED 710`
- **Security:** CSP, rate limiting, input sanitization, secure headers, and `frame-ancestors` allowing `oshinsarin.in` (so the overlay can embed).

12. Environment variables (`.env`)

Variable	Used for
<code>FASHN_API_KEY</code>	Fashn.ai virtual try-on
<code>CLOUDINARY_CLOUD_NAME</code> / <code>_API_KEY</code> / <code>_API_SECRET</code>	Cloudinary image hosting
<code>SUPABASE_URL</code> / <code>SUPABASE_ANON_KEY</code> / <code>SUPABASE_SERVICE_KEY</code>	Supabase
<code>SUPABASE_DB_PASSWORD</code>	Postgres connection
<code>ALLOWED_ORIGINS</code>	CORS (includes oshinsarin.in)

Secrets live only in `.env` (git-ignored).

13. Running locally

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt
cd backend && ../.venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000
# App: http://127.0.0.1:8000/widget/
```

14. Cost structure

<u>Service</u>	<u>Cost</u>
<u>Fashn.ai (virtual try-on)</u>	<u>5 credits per try-on at 4K quality</u>
<u>Fashn.ai credit price</u>	<u>\$0.075/credit on-demand ($\approx$ ₹6.4) $\rightarrow$ $\approx$ ₹32 per try-on</u>
<u>Minimum top-up</u>	<u>100 credits = \$7.50 ($\approx$ ₹640) $\rightarrow$ $\sim$20 try-ons</u>
<u>Supabase / Cloudinary / Vercel</u>	<u>Free tier</u>
<u>AILab via RapidAPI</u>	<u>Per-call (optional hairstyle feature)</u>

The infrastructure runs on free tiers; the only per-use cost is the Fashn.ai try-on render. Cost scales directly with usage — you pay only when a shopper actually tries an outfit on. Lower resolution tiers reduce the per-try-on cost (2K = 4 credits, 1K fast = 1 credit).

15. Operator cheat-sheet

Task	Command
Update catalog from de-signer sheet	<code>python backend/merge_metadata.py "<sheet>.xlsx"</code> then <code>python backend/migrate_products.py</code>
See try-on usage	<code>query Supabase</code> <code>tryon_history</code>
View captured leads	<code>python backend/view_leads.py</code>
Export leads to Excel	<code>python backend/export_leads.py</code> → <code>data/leads.xlsx</code>
Check Fashn.ai credits	<code>GET https://api.fashn.ai/v1/credits</code> (Bearer key)
Deploy a change	<code>git push origin main</code> (Vercel auto-deploys)
Top up credits	https://app.fashn.aiSMARTPANTS PRESERVED 877SMARTPANTS PRESERVED 878